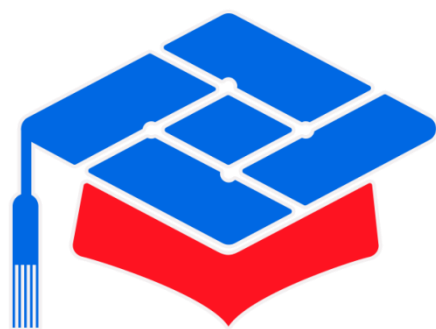




第十届
全国大学生集成电路创新创业大赛

报告类型：技术文档

参赛杯赛：“竞业达”企业命题

作品名称：五级流水 RISC-V 处理器设计与实现

队伍编号：CICC1001127

团队名称：未命名队

作品核心内容快速预览

本作品是一款采用 Verilog HDL 自主实现的 32 位 RISC-V 处理器，面向竞业达企业命题 FPGA 平台。处理器采用 IF、ID、EX、MEM、WB 五级顺序流水结构，并在数据访存通路中加入 MEM1 读数据寄存切分。当前 RTL 支持 RV32I 基础整数指令、RV32M 乘除法扩展、Zicsr 六类 CSR 指令，以及 ECALL、MRET 和非法指令异常处理。

表 0-1 作品技术要点快速索引

技术主题	实现要点	说明
流水组织	IF、ID、EX、MEM、WB 五级顺序流水；级间寄存器保存指令上下文	数据、控制与 PC 同步推进
数据相关	EX/MEM1/MEM/WB 多级前递；Load 相关时冻结前端并注入气泡	消费者取得最新可用结果
控制流	16 项两位 BHT；JAL/JALR 预测跳转；EX 级确认真实方向	错误路径由 ctrl 冲刷
多周期运算	MDU 锁存操作数；乘法短固定延迟；除法 32 轮迭代	busy 期间保持执行上下文
CSR/异常	支持 Zicsr、ECALL、MRET、非法指令异常	mstatus、mepc、mtvec、mcause 按 M 模式使用
平台接口	组合 IROM；perip 数据侧由 MEM1 寄存读数据	适配竞赛平台固定延迟接口

本文档按项目背景、总体架构、指令执行通路、数据相关控制、前端取指、执行与访存、CSR/异常、平台接口和 RTL 模块组织展开，重点说明提交版 RTL 的结构和功能实现。

目录

作品核心内容快速预览.....	2
第一章 项目背景、设计目标与技术路线.....	5
1.1 项目背景与平台约束	5
1.2 设计目标与实现内容	5
1.3 总体技术路线	5
第二章 总体架构与流水线组织.....	6
2.1 整机结构与模块职责	6
2.2 五级流水及指令上下文	6
2.3 全局控制原则与正确性约束	6
第三章 指令执行通路.....	7
3.1 算术逻辑与立即数类指令	7
3.2 Load/Store 数据通路.....	7
3.3 分支、JAL 与 JALR	8
3.4 RV32M 与系统指令	8
第四章 数据相关与流水线控制.....	9
4.1 RAW 冒险与多级前递.....	9
4.2 Load-Use 停顿机制.....	9
4.3 保持、气泡与冲刷	9
4.4 同周期事件仲裁	9
第五章 前端取指与分支预测.....	10
5.1 取指、预译码与下一 PC.....	10
5.2 16 项两位 BHT.....	10
5.3 JALR 基址前递与等待	11
5.4 EX 确认与错误恢复.....	11
第六章 执行、乘除法、访存与写回.....	12
6.1 EX 阶段功能汇聚.....	12
6.2 MDU 生命周期与除法迭代	12
6.3 访存接口与数据扩展	12
6.4 写回与副作用收口	12
第七章 CSR 与精确异常	14
7.1 CSR 读改写语义.....	14
7.2 异常进入与 MRET 返回.....	14

7.3 精确异常成立条件	14
第八章 FPGA 平台接口与存储器连接	15
8.1 IROM 与 perip 接口	15
8.2 固定延迟接口连接方式	15
8.3 主要结构资源	15
第九章 关键场景处理机制	16
9.1 典型事件的跨模块协同	16
9.2 相关场景处理	16
9.3 RTL 组织方式	16
第十章 RTL 文件组织与模块对应	18
10.1 RTL 模块分组	18
10.2 关键控制信号	18
10.3 提交版本实现范围	18
第十一章 仿真与测试报告	20
11.1 仿真平台与测试方法	20
11.2 功能测试结果	20
11.3 性能测试结果	21
附录 A 模块职责与源文件索引	25
附录 B 关键接口与控制信号	25
附录 C 指令、CSR 与编码支持范围	26
附录 D 指令扩展与模块对应	26

第一章 项目背景、设计目标与技术路线

1.1 项目背景与平台约束

本项目要求在官方 FPGA 工程和固定端口条件下实现自主 CPU。与算法模块不同，CPU 的正确性由多条指令在时间上的相互作用决定：一条指令可能正在写回，后一条正在访存，再后一条正在执行分支，前端同时还在预测新的取指地址。任何一级的数据或控制错位，都可能表现为偶发寄存器错误、重复 Store、错误跳转或异常返回失败。

平台提供组合读取的指令 ROM 和统一数据/外设接口，没有 valid/ready 握手。提交版 RTL 保持 IROM 组合取指方式，数据侧由 mem1 对 perip_rdata 进行时序采样，并在后一拍形成 Load 的符号/零扩展结果。整体结构采用五级顺序流水和集中控制。

1.2 设计目标与实现内容

表 1-1 设计目标及 RTL 对应关系

目标	实现内容	主要模块
完整指令执行	RV32I、RV32M、Zicsr、Zba/Zbb/Zbc/Zbs/Zbkb/Zbkx、ECALL/MRET	id、ex、mdu、csr_unit、Data_Hazard_unit
流水线控制	级间寄存器、前递、停顿、冲刷	if_id、id_ex、ex_mem1、mem1_wb、Data_Hazard_unit、ctrl
控制流处理	BHT 方向预测、JAL/JALR 预测、EX 恢复	ifetch、bpu、pc_reg、ex
平台接口	IROM 取指、perip Load/Store、MEM1 读数据寄存	myCPU、ifetch、mem1、wb

当前 RTL 的实现内容包括五级流水线、固定延迟平台接口、阻塞式 MDU、基础 CSR/trap 处理、BHT 方向预测和 Zb 位操作扩展。FENCE、FENCE.I 和 EBREAK 按无副作用指令处理。

1.3 总体技术路线

需求与 ISA 解析 → 基础数据通路 → 五级流水与 MEM1 访存切分 → 前递与停顿 → MDU/CSR/异常 → BPU 与顶层接口整合

各功能模块统一接入停顿、冲刷和重定向控制。BPU 的预测信息随指令进入流水，EX 给出真实跳转结果；MDU 通过 busy/done 保持流水状态；CSR 和异常通过统一重定向链更新 PC。

第二章 总体架构与流水线组织

2.1 整机结构与模块职责

PC/预测 → IF 取指 → IF/ID → ID 译码/读寄存器 → ID/EX → EX 执行/分支/MDU/CSR → EX/MEM1 → MEM1 访问并采样数据 → MEM/WB → WB 写回

表 2-1 五级流水职责

阶段	核心工作	主要输出
IF	保存/更新 PC，访问 IROM，预译码分支、JAL、JALR	指令、PC、预测方向和目标
ID	完整译码、立即数生成、寄存器读取、相关检测	操作数、目的寄存器、控制字段
EX	ALU、分支比较、目标计算、MDU、CSR/异常	执行结果、访存请求、重定向
MEM	驱动 perip，寄存 Load 读数据并完成扩展	寄存后的 Load 结果或 Store 副作用
WB	屏蔽 x0 写入，更新通用寄存器堆	架构可见寄存器状态

myCPU 顶层负责连接各级和统一控制信号。模块划分遵循单一职责：`pc_reg` 只选择下一 PC，`ifetch` 只做取指和早期识别，`id` 只做组合译码，`Data_Hazard_unit` 集中判断相关，`ex` 汇聚真实执行语义，`ctrl` 负责冲刷控制。这样能够避免同一规则散落在多个模块中。

2.2 五级流水及指令上下文

流水寄存器保存的不是若干互不相关的信号，而是一份完整“指令上下文”：指令编码、PC、源操作数、目的寄存器、写使能、访存属性、分支属性、系统操作和预测信息必须属于同一条指令并同步推进。若控制字段比数据早一拍或晚一拍，处理器可能把一条指令的结果写到另一条指令的目的寄存器。

当前设计没有单独 `valid` 位，而是用 `NOP` 编码和写使能、访存使能清零表示气泡。该方法适合小型顺序流水，但要求冲刷时所有可能产生副作用的控制字段同时归零。级间寄存器因此既是时序切分点，也是错误路径隔离点。

2.3 全局控制原则与正确性约束

表 2-2 PC 选择优先级

优先级	事件	处理
1	复位	PC 置 0x8000_0000
2	EX/系统重定向	采用已确认的真实地址
3	BPU 等待或流水停顿	保持当前 PC
4	预测跳转	采用预测目标
5	普通情况	PC+4

流水线控制主要围绕四类同步关系展开：数据、控制和 PC 同步推进；消费者使用最新可用结果；寄存器写、Store 和 CSR 更新只在对应指令有效时发生；重定向清除更年轻的错误路径指令。

表 2-3 指令上下文的关键字段及失配风险

字段类别	代表内容	若不同步可能出现的错误
身份信息	指令编码、PC、预测 PC	错误训练 BHT、异常 mepc 指向错误指令
数据字段	rs1/rs2 值、立即数、Store 数据	ALU 使用旧值或 Store 写入错误数据
控制字段	alu_op、reg_wen、is_load/store、system_op	结果写错目的地或产生不应有的副作用
目的信息	rd 地址、访存宽度、CSR 地址	写错寄存器、扩展方式错误或访问错误 CSR

从时序角度看，IF/ID、ID/EX、EX/MEM1、MEM1 内部数据寄存以及 MEM/WB 共同形成责任交接点。每个交接点都应同时回答三个问题：当前内容是否有效、下一拍是否允许推进、发生异常或冲刷时应写入什么安全值。若只保存数据而不保存相应控制，或者只清除写使能而遗漏 Store/CSR 属性，流水线看似继续运行，却会在更晚阶段产生难以定位的副作用。

第三章 指令执行通路

3.1 算术逻辑与立即数类指令

id 模块从 opcode、funct3、funct7、rs1、rs2、rd 和 shamt 字段确定指令类别，并按 I、R、B、J、U、Load、Store 和 SYSTEM 格式生成立即数。ID/EX 同时锁存操作数和 ALU 操作码；EX 通过统一组合运算器完成加减、比较、逻辑以及移位操作。LUI 直接选择高位立即数，AUIPC 使用 PC 与 U 型立即数相加，JAL/JALR 的链接值为 PC+4。

普通 ALU 指令的结果在 EX 末端即可确定，因此后一条依赖指令无需等待写回，可由前递网络直接取得 EX 结果。WB 仍是架构状态的唯一通用寄存器写入口，前递只是临时绕过写回延迟，不改变程序可见的提交顺序。

3.2 Load/Store 数据通路

Load 与 Store 都在 EX 计算有效地址 base+offset。Store 的 rs2 数据和宽度掩码随请求进入 MEM1，并通过 perip_wen、perip_mask 和 perip_wdata 对外产生写副作用；Load 则输出读地址，mem1 在时钟沿采样 perip_rdata，并在后一拍根据 funct3 完成 LB/LBU、LH/LHU、LW 的符号或零扩展，再经 MEM/WB 写回。

Load 的关键差异是结果不在 EX 形成，EX 中只有地址。提交版通过 mem1 内部寄存器锁存读数据，使 Load 结果从已寄存的 MEM 数据侧进入前递/写回链，避免直接使用外部组合读数据驱动执行级。当前统一数据接口仍没有 ready，默认请求后一拍数据可由 mem1 寄

存得到；若未来加入 Cache 或 AXI，Load 必须增加请求有效位、响应匹配和可变延迟保持机制。

3.3 分支、JAL 与 JALR

条件分支在 EX 根据 funct3 选择有符号/无符号比较，真实目标为 PC 加 B 型立即数；不跳转时正确后继为 PC+4。JAL 的目标由 PC 与 J 型立即数相加，JALR 的目标为 rs1+I 型立即数并清零最低位。JAL/JALR 同时将 PC+4 写入 rd。

预测信息在 IF 产生后随指令保存到 ID/EX。EX 比较真实 taken 与预测 taken；不一致时产生重定向和正确地址，ctrl 将 IF/ID、ID/EX 中的年轻指令置为安全气泡。

3.4 RV32M 与系统指令

RV32M 指令在 ID 识别后由 EX 内的 mdu 执行。操作数和操作码在 start 时锁存，busy 期间前端和 ID/EX 保持，done 到来后结果才能打开写使能。CSR 指令则由 csr_unit 读取旧 CSR 值作为 rd 写回数据，同时按照 CSRRW/CSRRS/CSRRC 及立即数变体计算新值。ECALL、MRET 和非法指令不走普通下一 PC，而是通过 system_redirect 进入统一重定向链。

表 3-1 代表性指令的逐级行为

指令类型	IF/ID	EX	MEM	WB/架构副作用
ADD/逻辑	取指并读取两个源寄存器	完成 ALU 运算，结果可前递	MEM1/MEM 透传结果	写 rd
Load	译码地址基址与偏移	计算有效地址	采样读数据并扩展	写 rd
Store	读取基址与待写数据	计算有效地址	驱动写地址、mask、wdata	无通用寄存器写回
条件分支	形成预测方向和预测后继	比较条件并确认真实后继	无数据运算	无寄存器副作用
JAL/JALR	预测跳转并携带目标信息	确认目标，同时形成 PC+4	MEM1/MEM 透传链接值	写 rd
CSR	译码 CSR 操作与地址	读取旧值、更新 CSR 或重定向	MEM1/MEM 透传旧值	必要时写 rd

不同指令的“完成”含义并不相同：普通算术指令以 WB 写入 rd 为完成，Store 以 MEM 对外产生一次有效写为完成，分支以 EX 确认正确下一 PC 为完成，CSR 指令还包含内部状态寄存器更新。控制逻辑必须针对副作用发生阶段抑制错误路径，而不能仅依赖最终 WB 写使能。

第四章 数据相关与流水线控制

4.1 RAW 冒险与多级前递

顺序五级流水不会产生真正的 WAW/WAR 乱序问题，主要风险是 RAW：消费者在生产者写回前读取同一寄存器。提交版 Data_Hazard_unit 比较 ID 的有效 rs1/rs2 与 EX、MEM1、MEM、WB 的目的寄存器，排除 x0 后选择最新结果。优先级必须是 EX 高于 MEM1、MEM1 高于 MEM、MEM 高于 WB，因为距离消费者最近的生产者在程序顺序上更年轻。

表 4-1 前递可用性

生产者位置	结果类型	是否可直接前递	原因
EX	普通 ALU/JAL/CSR 读值	是	结果已在 EX 组合逻辑形成
MEM1	非 Load ALU/JAL/CSR	是	结果已随 EX/MEM1 保存，可由 MEM1 前递
MEM1	Load	否	数据刚由 mem1 采样，需后一拍形成扩展结果
MEM/WB	任意可写回结果	是	Load 扩展结果或透传结果已确定

4.2 Load-Use 停顿机制

当 ID 的源寄存器依赖 EX 中尚未完成地址计算的 Load 时，冒险单元冻结 PC 与 IF/ID，使消费者停留在 ID；同时把 ID/EX 写成无副作用气泡，让老 Load 继续向后推进。对于分支、JALR 等会影响前端下一 PC 的消费者，提交版还会检查 MEM1/MEM 中的 Load，并在目标基址或比较操作数未就绪时继续等待。该方案把外部读数据从组合前递路径中移出，牺牲少量停顿换取更确定的时序。

周期 N：LW 在 EX、依赖指令在 ID → 冻结 PC/IF-ID，ID/EX 注入气泡 → Load 前进至 MEM1 并采样 perip_rdata → 数据进入 MEM/MEM-WB 可前递阶段 → 消费者进入 EX

4.3 保持、气泡与冲刷

保持表示“当前指令仍正确但暂时不能前进”，例如 MDU busy 时 ID/EX 必须保留当前乘除法指令；气泡表示“本周周期不允许新指令进入该级”，例如 Load-Use 时向 EX 注入 NOP；冲刷表示“该指令已经属于错误路径，必须永久删除”，例如误预测或异常后的 IF/ID、ID/EX。三者可能都表现为写入安全值或不更新寄存器，但语义和之后行为不同。

4.4 同周期事件仲裁

全局控制遵循真实重定向优先于等待和预测。若误预测与普通停顿同拍发生，PC 采用 EX 的正确地址；若 MDU 正在运行，ID/EX 保持当前乘除法指令；Load-Use 只影响年轻消费者，不阻塞更老指令完成。

表 4-2 ALU 连续相关的周期关系

周期	生产者位置	消费者位置	处理
N	EX	ID	比较 rd 与 rs，选 EX 结果
N+1	MEM	EX	消费者使用前递值执行
N+2	WB	MEM	两条指令继续顺序推进

表 4-3 Load-Use 的周期关系

周期	Load 位置	消费者位置	控制动作
N	EX	ID	检测 EX Load 相关；保持 PC/IF-ID，ID-EX 注入气泡
N+1	MEM	ID	检测 MEM Load 相关；继续保持，气泡向后推进
N+2	WB	ID→EX	寄存后的 Load 数据可前递，解除停顿并执行消费者

相关比较还必须结合“源寄存器是否真正被该指令使用”。例如 I 型算术通常不读取 rs2，LUI 不读取普通源寄存器，若仅按位域相等就停顿，会产生伪相关。当前 RTL 通过 ID_rs1/ID_rs2 有效标志和 x0 排除降低误判；未来扩展指令时，必须同步更新源操作数有效性定义。

第五章 前端取指与分支预测

5.1 取指、预译码与下一 PC

pc_reg 保存当前 PC，ifetch 将其直接送至 irom_addr，并对返回指令的 opcode 做早期识别，区分条件分支、JAL 和 JALR，同时生成相应立即数。早期预译码只提供预测所需的最小信息，完整合法性判断仍在 ID 完成。这样可以缩短预测启动时间，同时避免在 IF 重复全部译码逻辑。

5.2 16 项两位 BHT

条件分支使用 16 项、每项 2 位的 BHT，索引为 PC[5:2]。复位状态为 01（弱不跳转），最高位决定方向；实际分支在 EX 解析后，按照跳转向强跳转饱和、未跳转向强不跳转饱和。两位状态比单比特状态对循环退出更稳定：一次循环退出反转只降低置信度，不会立即完全改变长期预测倾向。

表 5-1 BHT 状态转移

状态	预测	实际跳转后的状态	实际不跳后的状态
00	不跳	01	00
01	不跳	10	00

状态	预测	实际跳转后的状态	实际不跳后的状态
10	跳转	11	01
11	跳转	11	10

该结构资源很小，但没有 tag，不同 PC 可能映射到同一项并相互污染；表项也只记录方向，不记录目标。因此它适合规则循环和稳定条件分支，不适合复杂相关模式或多目标间接跳转。

5.3 JALR 基址前递与等待

JAL/JALR 语义上必跳转，因此直接预测 taken。JAL 目标只依赖 PC 和立即数，ID 前端即可计算；JALR 目标依赖 rs1，若该寄存器正在 ID、EX、MEM1、MEM 或 WB 被更老指令写入，BPU 必须取得最新值。RTL 为 JALR 建立专用比较链：可用的 EX、非 Load MEM1、非 Load MEM 或 WB 结果直接前递；若值仍未产生，则 bpu_wait 保持 PC。

这一机制解决的是“预测目标的数据相关”，与普通 Data_Hazard_unit 的执行操作数相关职责不同。两套逻辑必须遵循相同的生产者优先级，否则前端可能用旧 rs1 计算目标，而执行级却用新值确认，造成必然重定向。

5.4 EX 确认与错误恢复

预测结果和预测时 PC 随指令进入流水。EX 获得真实分支条件后，若方向不同，real_jump_en 拉高：真实跳转时选择目标地址，真实不跳时选择 PC+4。pc_reg 将该地址置于预测和停顿之上，ctrl 同时清除更年轻的两级。训练与恢复都使用执行指令自身的 PC，防止前端已经前进后误更新其他 BHT 项。

表 5-2 一条条件分支的预测生命周期

阶段	处理内容	必须保持一致性
IF	用 PC[5:2] 查询 BHT，生成 taken 和预测后继	预测结果必须与该 PC 绑定
IF/ID→ID/EX	携带预测方向、预测地址和分支 PC	停顿时一起保持，冲刷时一起清除
EX	比较真实条件，计算真实目标/PC+4	使用同一条分支的上下文
恢复	若错误则覆盖 PC 并清除年轻级	真实重定向优先于预测和等待
训练	以执行分支 PC 更新对应 BHT 项	不能用当前前端 PC 代替训练 PC

条件分支在 EX 阶段解析。若预测错误，pc_reg 接收真实下一 PC，ctrl 清除 IF/ID 和 ID/EX 中的年轻指令，使取指回到正确路径。

JALR 的目标地址依赖 rs1。bpu 对 rs1 建立前递和等待链，能取得最新值时直接计算目标；若依赖的 Load 数据尚未进入可用阶段，则通过 bpu_wait 保持前端。

第六章 执行、乘法、访存与写回

6.1 EX 阶段功能汇聚

EX 是处理器的语义确认点：普通指令由 ALU 产生结果，分支在此比较并计算真实目标，Load/Store 在此生成地址，MDU 在此启动，CSR/异常在此决定是否重定向。功能汇聚便于统一控制，但也可能形成关键路径，因此设计通过 ID/EX、EX/MEM 寄存器把译码和访存分隔开。

6.2 MDU 生命周期与除法迭代

mdu 在 start 且非 busy 时锁存 op、src1、src2。乘法使用有符号×有符号、无符号×无符号和有符号×无符号的扩展乘积，根据 MUL/MULH/MULHSU/MULHU 选择低 32 位或高 32 位；随后在短固定延迟内产生 done。除法采用恢复余数算法，每拍移入一位被除数，比较并条件减除数，同时生成一位商，32 轮后输出商或余数。

DIV/REM 的符号通过绝对值运算和结果符号恢复处理；除数为 0 时，商返回全 1、余数返回被除数； $-2^{31} \div -1$ 溢出按 ISA 规定返回 -2^{31} ，余数为 0。mdu_stall 在结果完成前保持前级，rd_wen 只有在 done 后才开放，避免重复写回。

6.3 访存接口与数据扩展

EX/MEM1 保存地址、写数据、宽度和写使能，mem1 将其映射到 perip 接口。Store 通过 perip_wen=1 产生写入；Load 通过 perip_wen=0 给出读地址，mem1 在时钟沿采样 perip_rdata，并依据原始 funct3 在后一拍完成字节/半字的符号或零扩展。perip_mask 使用 00、01、10 表示字节、半字、字访问。

6.4 写回与副作用收口

mem1_wb 保存最终数据和目的寄存器，wb 在写回前屏蔽 rd=x0。通用寄存器的架构更新统一在 WB 发生，使前递网络、异常恢复和波形调试都可以围绕单一写入口分析。Store 和 CSR 的副作用分别发生在 MEM 和 CSR 时序逻辑中，因此控制信号必须确保被冲刷指令到达这些位置前已失效。

表 6-1 MDU 控制状态与流水线动作

状态	MDU 内部行为	流水线行为	风险控制
空闲	等待 start	普通推进	只在 is_mdu 且非 busy/done 时启动
锁存	保存 op/src1/src2	ID/EX 保持当前指令	防止下一条操作数覆盖
迭代/计算	乘积选择或逐轮除法	前端冻结，老后级可完成	禁止重复 start 与提前写回
完成	result 有效、done 脉冲	开放 rd_wen，随后恢复推进	结果只提交一次

表 6-2 Load/Store 宽度与数据处理

funct3/类型	访问宽度	Load 处理	Store 处理
LB/SB	8 bit	bit7 符号扩展	写低 8 bit, mask=字节
LBU	8 bit	零扩展	—
LH/SH	16 bit	bit15 符号扩展	写低 16 bit, mask=半字
LHU	16 bit	零扩展	—
LW/SW	32 bit	原值透传	写 32 bit, mask=字

perip_mask 使用平台约定编码。Store 的地址、掩码和写数据由 mem1 同周期输出；Load 读数据在 mem1 时钟沿采样，并按访问类型完成字节、半字或字扩展。

第七章 CSR 与精确异常

7.1 CSR 读改写语义

csr_unit 实现 mstatus、mtvec、mscratch、mepc 和 mcause。CSR 指令先组合读取旧值并送回 rd，再根据操作类型计算新值：CSRRW 直接覆盖，CSRRS 按位置位，CSRRC 按位清零；立即数版本使用 zimm。CSRRS/CSRRC 在源操作数为零时只读不写，未知地址产生 illegal_csr。

表 7-1 已实现 CSR

CSR	地址	作用
mstatus	0x300	保存 MIE/MPIE 等机器状态
mtvec	0x305	异常入口基址，按 Direct 模式使用
mscratch	0x340	异常处理软件临时寄存器
mepc	0x341	异常发生时保存当前指令 PC
mcause	0x342	记录非法指令或 ECALL 原因

7.2 异常进入与 MRET 返回

非法指令/非法 CSR 的 mcause 为 2，ECALL 的 mcause 为 11。异常时 csr_unit 保存当前 PC 到 mepc，将 MIE 复制到 MPIE 并清零 MIE，然后把 PC 重定向到 mtvec 对齐地址。MRET 将 MPIE 恢复到 MIE、把 MPIE 置 1，并跳转到 mepc 对齐地址。异常优先级高于 MRET 和普通 CSR 写，防止同拍状态被覆盖。

7.3 精确异常成立条件

精确异常要求异常之前的老指令已经或仍可按顺序完成，异常指令及其之后的年轻指令不得产生错误副作用。当前设计在 EX 识别异常：EX/MEM、MEM/WB 中的老指令继续推进，IF/ID 与 ID/EX 的年轻指令被冲刷；异常指令的通用寄存器写使能被关闭。由于处理器顺序发射、顺序完成，没有年轻指令提前提交，因此可以维护精确架构状态。

表 7-2 六类 CSR 指令语义

指令	rd 获得	CSR 新值	源为零时
CSRRW	旧 CSR	rs1	仍执行覆盖
CSRRS	旧 CSR	旧值 OR rs1	rs1=x0 时只读
CSRRC	旧 CSR	旧值 AND NOT rs1	rs1=x0 时只读
CSRRWI	旧 CSR	zimm	仍执行覆盖
CSRRSI	旧 CSR	旧值 OR zimm	zimm=0 时只读
CSRRCI	旧 CSR	旧值 AND NOT zimm	zimm=0 时只读

表 7-3 异常进入与返回的状态变化

事件	mepc/mcause	mstatus	下一 PC
非法指令/非法 CSR	mepc=当前 PC, mcause=2	MPIE←MIE, MIE←0	mtvec & ~3
ECALL	mepc=当前 PC, mcause=11	MPIE←MIE, MIE←0	mtvec & ~3
MRET	保持 mepc/mcause	MIE←MPIE, MPIE←1	mepc & ~1

异常识别位于 EX，意味着异常指令之前的指令可能已经在 MEM 或 WB。冲刷范围必须以程序年龄为依据：只清除 IF/ID 与 ID/EX，不回滚已经更老的状态；同时关闭异常指令自身的 rd_wen，防止它在重定向后仍写回。

第八章 FPGA 平台接口与存储器连接

8.1 IROM 与 perip 接口

顶层 irom_addr 直接来自 PC，irom_data 作为本周期指令进入 ifetch。数据侧通过 perip_addr、perip_wen、perip_mask、perip_wdata 和 perip_rdata 统一访问 DRAM 与外设。提交版 mem1 会寄存 Load 读数据，但接口本身没有请求有效、等待或响应有效信号，因此 CPU 仍不保存可变延迟事务，也不允许存储器任意延迟返回。

8.2 固定延迟接口连接方式

指令侧使用组合 IROM，irom_addr 由 PC 给出，irom_data 进入 ifetch。数据侧使用统一 perip 接口，mem1 对 perip_rdata 做一级寄存，Load 扩展结果再进入写回链。

8.3 主要结构资源

表 8-1 主要结构资源

结构	主要资源	实现方式
寄存器堆与流水寄存器	FF/LUT 或分布式 RAM	保存通用寄存器和各级指令上下文
BHT	少量 FF 或分布式 RAM	PC[5:2] 索引 16 项两位饱和计数器
前递网络	寄存器比较器与多路选择器	按 EX→MEM1→MEM→WB 优先级选择源操作数
MDU	乘法器/DSP、除法迭代寄存器	乘法组合乘积后短延迟完成，除法 32 轮迭代
MEM1 数据接口	地址、控制和读数据寄存器	Store 直连 perip，Load 采样 perip_rdata 后扩展

主要结构资源集中在寄存器堆与流水寄存器、BHT、前递比较网络、MDU 以及数据侧 MEM1 寄存接口。乘法运算可由综合工具映射到 FPGA DSP 资源，除法使用迭代寄存器实现。

上述结构均在顶层 myCPU 中连接，控制信号由 ctrl、Data_Hazard_unit、bpu、ex 和 csr_unit 共同产生。

第九章 关键场景处理机制

9.1 典型事件的跨模块协同

表 9-1 关键场景的模块协同

场景	检测位置	控制动作	处理结果
ALU 连续相关	Data_Hazard_unit	EX/MEM1/MEM/WB 前递，不停顿	消费者获得最新生产者结果
Load-Use	Data_Hazard_unit	保持 PC/IF-ID，ID-EX 注入气泡	Load 数据进入可前递阶段后继续执行
JALR 基址未就绪	bpu	可前递则计算目标，否则 bpu_wait	rs1 最新值可用后更新 PC
分支误预测	ex/ctrl/pc_reg	真实地址覆盖预测，清除年轻级	正确路径重新取指
MDU busy	ex/mdu	保持前端与 ID/EX	done 后写回结果
异常与返回	csr_unit/ex/ctrl	保存 CSR、重定向 mtvec 或 mepc、冲刷流水级	进入处理程序或从 MRET 返回

9.2 相关场景处理

连续 RAW 相关由 Data_Hazard_unit 比较各级目的寄存器并选择最新前递数据；Load-Use 相关通过保持 PC/IF-ID 和向 ID/EX 注入气泡处理；JALR 基址相关由 bpu 的专用前递和等待链处理。

多周期乘除法由 mdu 锁存操作数和操作类型，busy 期间保持流水，done 后开放写回。分支和异常通过统一真实重定向链更新 PC，并清除更年轻的流水级。

9.3 RTL 组织方式

RTL 按功能划分为取指、译码、执行、访存、写回、相关控制、分支预测和 CSR/异常等模块。顶层 myCPU 负责连接各模块的数据通路、控制通路和平台接口。

表 9-2 关键场景处理方式

场景	涉及模块	处理方式	相关信号
Load 后消费者	Data_Hazard_unit、id_ex、mem1	EX Load 相关时保持前端并注入气泡	PC_stall、 IF_ID_stall、 ID_EX_nop、is_load
JALR 基址相关	bpu、Data_Hazard_unit	建立 rs1 专用前递与等待链	bpu_wait、 forward_rD、 prdt_jump_addr

场景	涉及模块	处理方式	相关信号
MDU 多周期执行	ex、mdu、ctrl	锁存输入并用 busy/done 控制流水保持	mdu_stall 、 mdu_busy、mdu_done
分支恢复	ex、ctrl、pc_reg	真实地址覆盖预测并冲刷年轻级	real_jump_en 、 real_jump_addr 、 ctrl_hold_flag
异常返回	csr_unit、ex、ctrl	异常进入 mtvec，MRET 返回 mepc	system_redirect 、 mepc 、 mcause 、 mstatus

第十章 RTL 文件组织与模块对应

10.1 RTL 模块分组

提交版 RTL 按功能可分为取指与 PC、译码与级间寄存、执行与 MDU、访存与写回、相关控制、分支预测、CSR/异常和通用寄存器等模块组。

表 10-1 RTL 模块分组

模块组	文件	主要功能
顶层与取指	myCPU.v、pc_reg.v、ifetch.v	连接平台接口、维护 PC、取指并携带预测信息
译码与级间寄存	id.v、if_id.v、id_ex.v、ex_mem1.v、mem1_wb.v	生成控制字段并在流水级间保存指令上下文
执行与乘除法	ex.v、mdu.v	完成 ALU、分支比较、MDU 和系统操作入口
访存与写回	mem1.v、wb.v	产生 perip 访问，寄存 Load 数据并写回寄存器堆
控制与预测	Data_Hazard_unit.v、ctrl.v、bpu.v	处理前递、停顿、冲刷、分支预测和 JALR 等待
状态寄存器与公共定义	regs.v、csr_unit.v、defines.v、dff_set.v	实现 GPR、CSR/trap、指令宏定义和通用寄存器单元

10.2 关键控制信号

关键控制信号包括 PC 选择与重定向、流水保持与气泡、前递选择、MDU busy/done、CSR 系统重定向以及 perip 访存控制。

10.3 提交版本实现范围

提交版本实现 RV32I、RV32M、Zicsr、ECALL、MRET 以及非法指令异常。数据存储接口采用固定延迟 perip 连接方式，Load 数据由 mem1 寄存后进入写回链。

表 10-2 提交版本实现范围

类别	实现内容	对应模块
基础整数指令	RV32I 算术逻辑、比较、移位、跳转、分支、Load/Store	id.v、ex.v、mem1.v、wb.v
乘除法扩展	RV32M 的 MUL、MULH、MULHSU、MULHU、DIV、DIVU、REM、REMU	id.v、ex.v、mdu.v
CSR 与 trap	Zicsr 六类 CSR 指令、ECALL、MRET、非法指令/非法 CSR	id.v、csr_unit.v、ex.v、ctrl.v
流水线控制	多级前递、Load-Use 停顿、MDU 保持、分支/异常冲刷	Data_Hazard_unit.v、ctrl.v、bpu.v、级间寄存器

类别	实现内容	对应模块
平台接口	组合 IROM、固定延迟 perip、MEM1 Load 读数 数据寄存	myCPU.v、ifetch.v、mem1.v

本章列出的模块和信号均与 `ready_to_commit_rtl` 中的源文件对应，便于根据正文说明阅读 RTL 代码。

第十一章 仿真与测试报告

本章对处理器进行 RTL 行为级功能验证与性能采样。功能验证包括：RV32I 指令集。结合当前 CPU 译码与执行模块，测试可按指令类型分组：R 型算术逻辑、I 型立即数算术逻辑、Load/Store、Branch、Jump、U 型指令。每条指令执行后，程序将实际结果与预期结果进行比较；若比较失败，则跳转到对应失败分支，将失败编号写入状态地址；若全部通过，则写入 PASS 状态码 1。

11.1 仿真平台与测试方法

仿真包包含两个主要 testbench：tb_ready_cpu_core_smoke.v 用于快速验证 CPU 核心指令与异常链路；tb_ready_cpu_func_perf.v 加载 irom-v2.mem 与 dram.mem，建立 IROM、DRAM 和 MMIO 外设模型，统计退休指令数、CPI、MDU 停顿、Load-Use 停顿、BPU 等待与重定向次数，并输出波形数据库。

表 11-1 仿真文件与用途

文件	作用	报告使用方式
run_xsim_ready_cpu.tcl	一键编译、展开并运行 smoke 与功能/性能仿真	复现实验入口
tb_ready_cpu_core_smoke.v	内嵌短程序，检查 RV32I、RV32M、Zicsr、ECALL、MRET	功能正确性冒烟测试
tb_ready_cpu_func_perf.v	加载 withMext 程序与数据存储器，建立 MMIO 模型并统计性能计数	功能运行与性能采样
waves_core_smoke.tcl / waves_ready_cpu.tcl	记录 testbench 与 CPU 内部信号波形	调试与结果留存
mem/irom-v2.mem、mem/dram.mem	由 COE 转换得到的指令和数据初始化文件	程序输入

11.2 功能测试结果

核心冒烟测试在复位后执行乘法、除法、CSR 读写、ECALL 异常进入和 MRET 返回程序，并检查通用寄存器结果。XSim 输出 PASS core integration: RV32I/RV32M/Zicsr/ECALL/MRET，说明核心指令路径、MDU、CSR/Trap 和异常返回链路均通过该测试。

```
run all
PASS core integration: RV32I/RV32M/Zicsr/ECALL/MRET
$finish called at time : 1026 ns
```

图 1-控制台输出功能测试通过

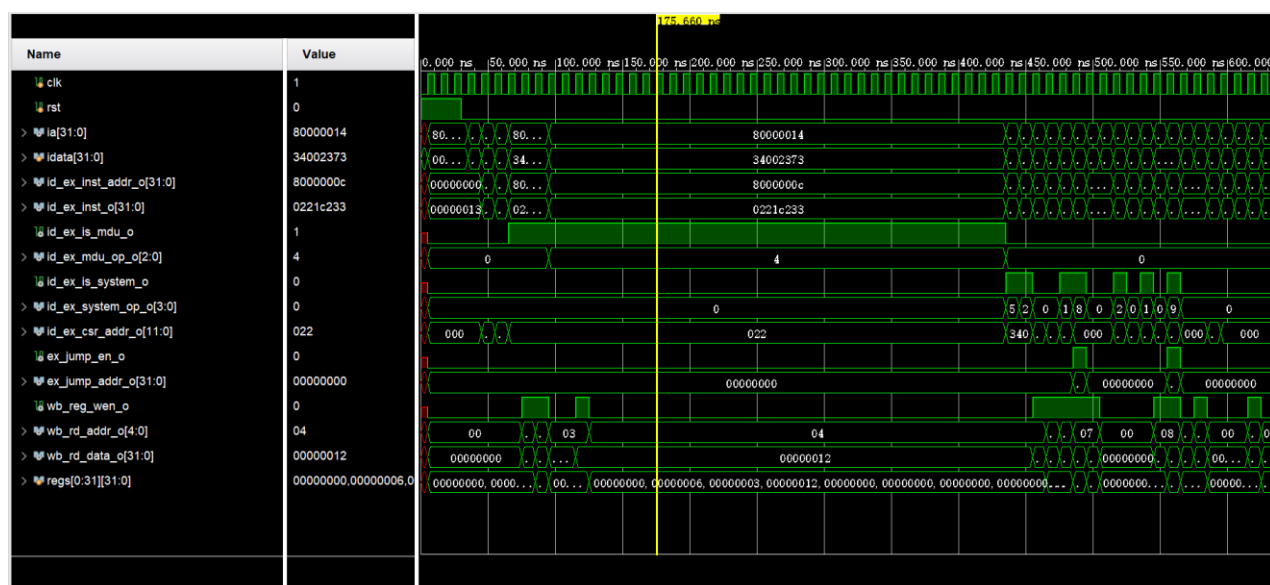


图 2-指令执行与最终状态写入波形图

表 11-2 功能覆盖与结论

测试项	检查内容	结果
RV32I 基础执行	ADDI、AUIPC、顺序执行与跳转自循环	通过
RV32M	MUL 得到 18，DIV 得到 6	通过
Zicsr	CSRRWI 写 mscratch，CSRRS 读回	通过
异常入口	ECALL 跳转到 mtvec 指定 handler	通过
异常返回	handler 修改 mepc 后 MRET 返回后续指令	通过

11.3 性能测试结果

功能/性能仿真使用 irom-v2.mem 和 dram.mem，在 200000 个周期的有界采样窗口内运行。仿真未作为失败超时处理，而是在采样窗口到达后输出统计摘要，用于观察当前程序负载下的流水线行为和性能特征。运行过程中可见 MMIO 事件：SEG 写入 0x34100000，计数器启动写入 0x80000000，LED 写入 0x00020000。

表 11-3 性能统计摘要

指标	数值	说明
采样周期数	200000	MAX_CYCLES 采样窗口
退休指令数	135976	按 EX 完成指令统计
CPI	1.471	cycles / retired

ALU / Load / Store	76164 / 33874 / 8714	基础计算与访存指令
Branch / Jump / System	4421 / 187 / 28	控制流与系统指令
MDU 指令数	12588	mul=8371, mulh=4201, div=8, rem=8
MDU 停顿周期	25486	最长连续 mdu_stall 为 33 周期
Load-Use 停顿	33928	与 data_stall_cycles 一致
BPU 等待周期	16	主要来自 JALR 基址依赖
重定向次数	85	branch=81, system=4, jump=0

11.3.1 数据前递结果

在 add x23, x1, x2 后紧跟 add x24, x23, x1 的场景中，后一条指令读取的源寄存器依赖前一条指令尚未最终写回的目的寄存器。波形应显示 forward1 或 forward2 在相关周期拉高，并且前递数据与前一条指令的执行结果一致。最终 x24 的检查结果通过，说明前递路径能够避免不必要的长时间停顿。



图 3-数据前递 forward 图

该测试说明 CPU 对 EX/MEM/WB 阶段到后续指令的 RAW 相关具备处理能力，提高了连续

相关算术指令的执行效率。

11.3.2 Load-use 暂停结果

在 `lw x21, 0(x10)` 后紧跟 `addi x22, x21, 1` 的场景中，加载数据需要经过访存阶段才能返回。波形中应能观察到 `PC_stall`、`IF_ID_stall` 和 `ID_EX_nop` 拉高，表示流水线暂停取指和译码，并向执行阶段插入空操作。暂停结束后，相关指令继续执行，最终结果比较通过。

该测试说明 CPU 能够识别 load-use 冒险，并通过暂停和插入气泡保证结果正确。虽然该场景会增加 CPI，但属于流水线 CPU 中必要的正确性保护。

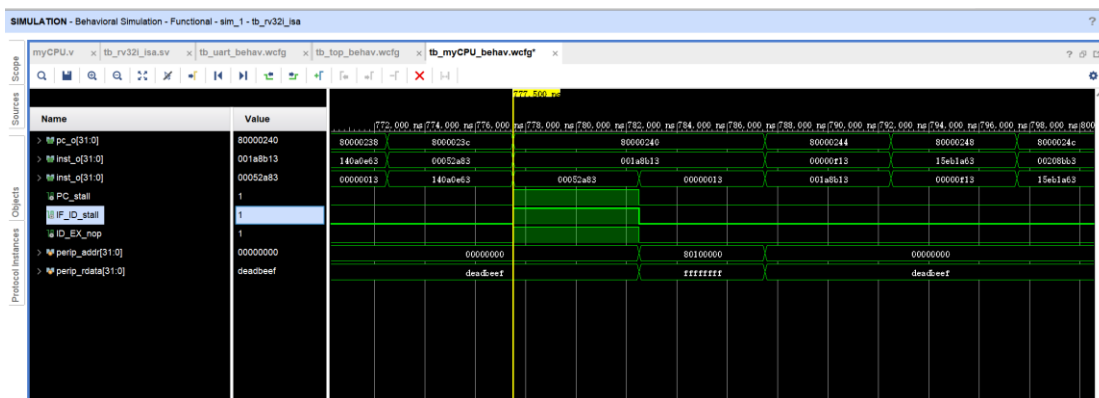


图 4-load-use 暂停图

11.3.3 分支跳转结果

在 Branch 和 Jump 测试片段中，PC 能够根据分支条件或跳转指令更新到目标地址。对于 JAL/JALR，链接寄存器写入返回地址；对于条件分支，条件成立时跳转到目标标签，条件不成立时继续顺序执行。波形中可观察 `ex_jump_en_o` 或 `ctrl_jump_en_o` 的变化，以及 PC 从当前地址切换到目标地址。

该结果说明 CPU 的控制冒险恢复机制能够保证程序控制流正确。结合 BPU 更新信号，可进一步说明分支预测表会在执行阶段根据真实结果更新。

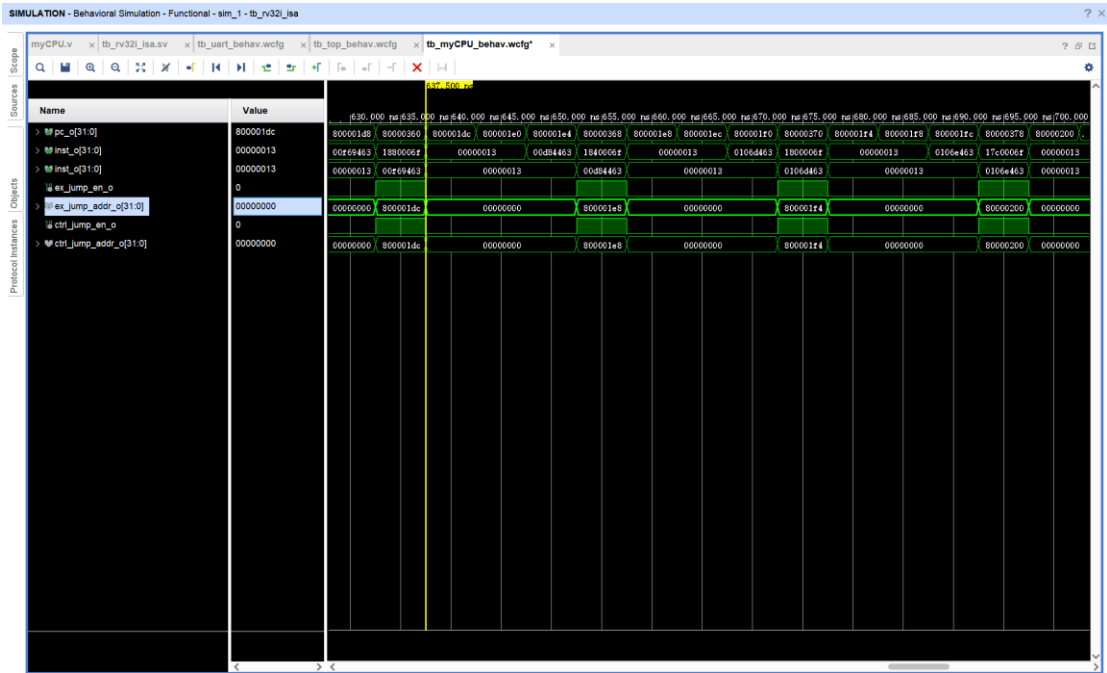


图 5-分支/跳转 PC 重定向图

附录 A 模块职责与源文件索引

表 A-1 RTL 模块索引

源文件/模块	职责	所在阶段
myCPU.v / myCPU	顶层连接、控制信号汇聚	全局
pc_reg.v / pc_reg	下一 PC 优先级与状态保存	IF
ifetch.v / ifetch	IROM 访问、控制流预译码	IF
if_id.v / if_id	保存指令、PC、预测信息	IF/ID
id.v / id	完整译码、立即数、控制生成	ID
regs.v / regs	32×32 寄存器堆及旁路读	ID/WB
id_ex.v / id_ex	保存执行上下文、接入前递	ID/EX
Data_Hazard_unit.v	RAW 检测、前递与 Load 停顿	ID/控制
bpu.v / bpu	16 项 BHT、JALR 目标处理	IF/控制
ex.v / ex	ALU、分支、地址、MDU/CSR 汇聚	EX
mdu.v / mdu	乘除法状态机	EX
csr_unit.v / csr_unit	CSR 与异常状态	EX
ex_mem1.v / ex_mem1	保存执行/访存结果	EX/MEM
mem1.v / mem1	perip 访问、Load 读数据寄存与扩展	MEM
mem1_wb.v / mem1_wb	保存最终写回数据	MEM/WB
wb.v / wb	寄存器写回与 x0 屏蔽	WB
ctrl.v / ctrl	重定向冲刷控制	控制
dff_set.v / dff_set	通用流水寄存单元	各级

附录 B 关键接口与控制信号

表 B-1 顶层平台接口

接口	方向/位宽	含义
cpu_clk / cpu_rst	输入 / 1	系统时钟、同步高电平复位
irom_addr / irom_data	输出 32 / 输入 32	组合取指地址与指令数据
perip_addr	输出 32	数据或外设访问地址
perip_wen	输出 1	1=Store, 0=Load
perip_mask	输出 2	00 字节、01 半字、10 字
perip_wdata / perip_rdata	输出 32 / 输入 32	写数据与读数据输入；Load 读数据在 mem1 采样

表 B-2 关键内部控制信号

信号	来源	作用
prdt_taken / 预测目标	bpu	决定前端预测下一 PC
bpu_wait	bpu	JALR 基址尚未可用时保持 PC
PC_stall / IF_ID_stall	Data_Hazard_unit	Load 相关时冻结前端
forward1/2、forward_rD1/2	Data_Hazard_unit	向 ID/EX 提供最新操作数
mdu_stall	ex/mdu	多周期运算期间保持上下文
real_jump_en / real_jump_addr	ex/csr_unit	误预测或异常的真实重定向
ctrl_hold_flag	ctrl	将年轻级写成安全气泡
rd_wen / rd_addr / rd_data	各级至 wb	最终通用寄存器写回链

附录 C 指令、CSR 与编码支持范围

表 C-1 指令支持范围

类别	支持内容
RV32I	LUI、AUIPC、JAL、JALR；6 类条件分支；LB/LH/LW/LBU/LHU；SB/SH/SW；整数立即数与寄存器算术、逻辑、比较和移位
RV32M	MUL、MULH、MULHSU、MULHU、DIV、DIVU、REM、REMU
Zicsr	CSRRW、CSRRS、CSRRC、CSRRWI、CSRRSI、CSRRCI
异常/返回	非法指令、非法 CSR、ECALL、MRET
无副作用处理	NOP；FENCE、FENCE.I、EBREAK 当前不执行完整体系结构语义

表 C-2 关键 CSR 地址

名称	地址	当前用途
mstatus	0x300	MIE/MPIE 状态保存与恢复
mtvec	0x305	异常入口
mscratch	0x340	软件临时数据
mepc	0x341	异常 PC
mcause	0x342	异常原因

附录 D 指令扩展与模块对应

表 D-1 指令扩展与模块对应

指令/功能类别	译码模块	执行/控制模块	说明
RV32I	id.v	ex.v、mem1.v、wb.v	基础整数、分支跳转、Load/Store 和写回

指令/功能类别	译码模块	执行/控制模块	说明
RV32M	id.v	ex.v、mdu.v	MUL/MULH/MULHSU/MULHU、DIV/DIVU、REM/REMU
Zicsr	id.v	csr_unit.v、ex.v	六类 CSR 指令读旧值并按规则写新值
Trap	id.v	csr_unit.v、ctrl.v、pc_reg.v	ECALL、MRET、非法指令/非法 CSR 重定向
数据相关	id.v、Data_Hazard_unit.v	ctrl.v、级间寄存器	前递、Load-Use 停顿和气泡注入
控制流	ifetch.v、bpu.v	ex.v、ctrl.v、pc_reg.v	BHT 方向预测、JAL/JALR 处理和真实重定向